



A Deep Learning Approach for Botnet Detection Using Raw Network Traffic Data

Mohaddeseh Shahhosseini¹ · Hoda Mashayekhi¹ · Mohsen Rezvani¹

Received: 7 September 2021 / Revised: 9 December 2021 / Accepted: 25 February 2022 /
Published online: 1 April 2022

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2022

Abstract

Botnets are considered to be one of the most serious cybersecurity threats in recent years. While botnets have been widely studied, they are constantly evolving, becoming more sophisticated and robust against detection systems. Current approaches of botnet detection commonly use manual feature engineering or analyze packet contents violating the privacy of users. Although some studies use raw packet bytes for botnet detection, this approach is rarely reported for the comprehensive ISCX botnet dataset. In this paper, we propose a deep learning-based network traffic analyzer for botnet detection, which automatically extracts the convenient features from raw packet data. The raw data is extracted only from the headers of the first few packets in a flow. The proposed approach lifts the costs of manual feature engineering, preserves user privacy, and offers early detection of malicious traffic. We further enrich the raw data with temporal information of packets and field correlations, to construct four different flows signatures. The evaluations are performed with the ISCX botnet dataset, which contains new botnet types in its test data. We show the effectiveness of botnet detection based on raw data, by comparing the performance of the proposed approach against several feature-based methods. The evaluation results also show that the proposed approach outperforms several state-of-the-art studies based on the same dataset, and provides high accuracy of 97.13% in classifying network traffic.

Keywords Botnet detection · Network traffic classification · Raw packets flow · Deep learning · Temporal information · Correlation

✉ Hoda Mashayekhi
hmashayekhi@shahroodut.ac.ir

¹ Faculty of Computer Engineering, Shahrood University of Technology, Shahrood, Iran

1 Introduction

Botnets are one of the most significant cyber security threats for all systems connected to the Internet [32]. Bots are malicious software programs, which infect many hosts and can create a network of compromised hosts to exploit them in some destructive activities, such as sending spam emails and running distributed denial-of-service (DDoS) attacks. The Botnet is a network of bots, being remotely controlled by at least one master host. An attacker uses the master host to transfer orders to the bots through command and control (C&C) channels [43]. Due to a large number of botnets and the collective malignant activities in these networks, they have become a serious threat to cyber security [43]. The threats posed by botnets include DDoS attacks, click fraud, information theft, resource consumption, spam transmission, and exploitation of other hosts to further spread the network [46]. The C&C communication channels can be centralized or distributed [38]. It is generally more difficult to detect attacks in decentralized C2C infrastructures, such as peer-to-peer (P2P) networks [8].

Botnet detection is a widely studied topic and different approaches have been proposed in this regard [2, 23, 41]. Some studies use signature-based approaches [44], which although being promising in detecting widely recognizable botnets, are unable to detect unknown botnets and their types, and fail if the traffic is encrypted. Anomaly-based identification approaches [2, 28] perform diagnostic actions by distinguishing abnormal traffic flows in the traffic based on normal behavior. These methods are restricted when bots mimic the communication patterns of regular hosts to prevent detection. Honeypot-based diagnostic approaches can only detect existing bots and perform poorly in real-time. Identification approaches based on specific protocols and structures [35] are intrinsically limited.

As the destructive capabilities of new attacks develop, traditional machine learning algorithms become less effective in counteracting the botnets [39]. In this regard, deep learning is a more promising approach to cover the weaknesses of common learning-based botnet detection methods. Deep learning networks can learn different features of the data, and use them to make decisions about unseen similar data. In recent years, due to various algorithmic innovations and significant improvements in computational capabilities, deep learning algorithms are enjoying widespread adoption in intrusion detection [39, 42].

The learning-based intrusion detection methods face some challenges in practice. Many current studies struggle to find a suitable set of traffic features. Although the selected features have a large impact on classification accuracy [2], it is difficult to provide general solid guidelines for choosing the effective features [45]. Figure 1 shows the general procedure of feature extraction from the network data in different botnet detection algorithms. Commonly, the raw network packets are divided into network flows distinguished by the source and destination addresses, and transmission protocol and ports. As each packet consists of header and payload parts, either or both of them may be used in feature extraction. In both cases, the appropriate features can be obtained manually and

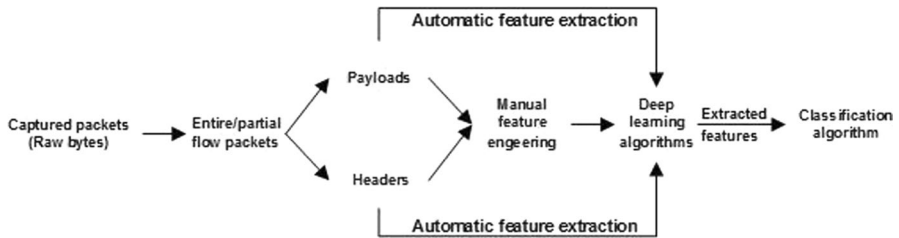


Fig. 1 The general procedure of feature extraction in botnet detection

experimentally, or automatically. The manual feature design and extraction poses computational costs for preprocessing traffic flows. It may also not be generalizable to different datasets. Other issues in such approaches are their low detection rate and their high false alarm rate, which should be improved [19]. These issues become more essential for unseen events and attack types, and demand evaluation of the proposed model to be performed with a comprehensive data set, covering malicious traffic generated by a variety of botnets [2]. In the intrusion detection context, a large portion of current studies use deep structures to learn a second level of features from the previously handcrafted features [7, 21, 29]. Deep learning algorithms have recently achieved great success in extracting appropriate features from data. However, when the features are initially limited and manually selected, not all of the relevant information may be provided to the deep learning algorithm.

Recent attention has focused on using representation learning [3] for intrusion detection, which enables automatic feature extraction from raw traffic data [13, 15, 22, 48] as shown in Fig. 1. Deep neural networks can be effective in this regard for replacing manual feature engineering. In the automatic feature extraction, the raw flow packets are given to the deep learning algorithm to extract the appropriate features. As the actual packet bytes are provided to the learning algorithm, diverse features may be extracted using the capabilities of deep learning methods, and used for botnet classification. Minimal preprocessing may also be included to increase the classification accuracy. The capacities of deep networks in directly deriving features from raw data are still largely unexposed in botnet detection. Although some studies [13, 15, 22, 48, 49] have moved on in this context, several challenges remain to be addressed. To provide early detection of the malicious traffic, the detection engine should decide based on the initial packets in a flow. This can aid in preventing the accomplishment of security attacks. Detection is preferred to be based on packet header data as using the payload may violate user privacy, while facing problems for encrypted traffic. Both the detection rate of the algorithm and the false alarm rate is also important to not be adversely affected by the feature extraction approach.

To address the above-mentioned challenges, in this paper, we propose a deep-learning-based algorithm for botnet detection in network traffic. The algorithm uses raw data from network flows and a Long short-term memory (LSTM) network to automatically extract the convenient features for classification. The raw data are extracted only from headers of the first few packets in each flow. This design lifts the

costs of manual feature engineering, protects privacy while offering early detection of malicious traffic. We further enrich the raw data with temporal information of packets and field correlations to improve the classification accuracy. Four different signatures are constructed for each flow and compared in the experiments. The evaluations are performed with the comprehensive ISCX botnet dataset, which contains 16 different botnet types, many of which are only seen in the test data. To the best of our knowledge, no previous work on the ISCX dataset uses raw flow data as model input. We compare the performance of the proposed approach against feature-based methods, and also other studies using the same dataset. The evaluation results show that the LSTM network can successfully extract features from raw data and the classifier produces a high accuracy, around 97.13% based on the extracted features. The highlights of this paper are summarized below:

- Proposing a deep-learning-based model for botnet detection
- Automatic feature extraction from raw network flows based on only the header information of packets, and the first few packets in a flow
- Proposing flow signatures, based on raw data, temporal flow aspects, and correlation information
- Evaluating and comparing the model performance using the comprehensive ISCX botnet dataset which has a large portion of unseen botnet types in its test data

In the rest of the paper, Sect. 2 describes the related research. In Sect. 3 the proposed botnet detection method is introduced. Evaluation of the proposed method is mentioned in Sect. 4, and Sect. 5 concludes the paper.

2 Related Work

Botnets are an important means of distributing new malicious software across a wide range of infected hosts and performing security attacks [38]. Over the past decades, researchers have introduced a variety of traditional machine learning and data mining methods to identify and track botnets. Miller and Busby-Earle [31] evaluated several sets of features for classifying the ISCX botnet dataset. A variety of supervised and unsupervised learning algorithms are used to evaluate the feature sets. Tan et al. [40] used the principles of multivariate correlation analysis to introduce an anomaly-based detection system to identify DDoS attacks. They tried to capture the behavior of traffic for detecting known and unknown attacks. Beigi et al. [2] used a set of flow-level features to describe and classify the ISCX dataset. Meghdouri et al. [30] propose a feature vector representing traffic at three levels: application, conversation, and endpoint behavior. Garre et al. [9] have proposed a method for infecting SSH-based botnets, and used machine learning (ML) techniques to develop a real-time detection model. Jagadeesan et al. [20] have developed a model for early botnet detection. An advanced support vector neural network is presented as a diagnostic model in this paper. To increase the accuracy of classification, the appropriate characteristics of traffic flows are selected from the data set. They investigate the

impact of encryption on feature spaces and detect attacks based on the Random Forest classifier.

Recently, deep learning algorithms have achieved significant success in various applications. The LSTM network is extensively used for in sequence learning tasks in different domains, and is successful in classifying long sequences [10]. It is also adopted in intrusion detection systems to improve system performance [24]. Yin et al. [44] created a generative adversarial network to oversample the labeled instances and perform classification. Hossain et al. [16] introduced a method to reduce the selected features from network flows, and used neural networks to detect the botnets. Hosseini et al. [17] proposed a method for detecting and comparing four botnet attack types, the IRC, HTTP, DNS, and P2P, based on CNN-LSTM network architecture.

In practical conditions, anomaly-based intrusion detection methods face several challenges. First, finding a suitable set of traffic features is difficult, as the selected features have a large impact on the botnet detection accuracy. Currently, there are no solid guidelines for discovering a set of reasonable features for traffic identification [45]. Another problem with anomaly-based intrusion detection methods is the high rate of false alarms [19]. Representation learning [3] is a promising approach to solve these two problems.

Representation learning, also called feature learning, is a technique that enables automatic feature extraction from raw data. Its biggest advantage is that it can replace manual feature engineering with direct extraction of convenient features from raw data. Deep neural networks (DNNs) are the most successful approaches in this regard and have significant results in different fields such as computer vision and natural language processing [11]. In the intrusion detection context, the majority of current studies, although using representation learning, learn features from previously handcrafted features. For example, Ma et al. [29] applied DNNs to detect the intrusive behaviors in the KDD99 data set. Niyaz et al. [21] studied Deep belief networks (DBNs) on the NSL-KDD data set for intrusion detection. Eesa et al. [7] have been able to reduce the wrong warning rate by choosing among different feature sets. Nevertheless, the power of deep networks in directly deriving features from raw data, as demonstrated in other fields of research, is largely unexposed in botnet detection. Therefore, these methods do not take full advantage of DNNs. In this study, we try to improve the accuracy of botnet detection by learning the features automatically.

Homayoun et al. [15] introduced the deep learning BotShark method, to analyze traffic on centralized and P2P bots. The proposed method consists of two deep structures, the first layer uses stacked Autoencoders for feature extraction, and the second layer uses a convolutional neural network (CNNs) for botnet detection. The extracted features include: (1) byte-based attributes, (2) time-based attributes, and (3) packet-based attributes. Kim et al. [22] introduce a combined and scalable intrusion detection system based on the Spark ML and Conv-LSTM networks. Hamed et al. [13] proposed a network-based intrusion detection system based on the recursive feature addition (RFA) feature selection method and the bigram technique. The bigram method is used to create a display of data content and use it to select features. Wang et al. [45] proposed HAST-IDS for intrusion detection. Both header

and payload data have been used to identify flow types. The features are extracted by CNN and LSTM algorithms. Yu et al. [48] used an encoder to automatically extract attributes for intrusion detection. Zhong et al. [49] proposed a hierarchical deep learning system for botnet detection. It uses both behavioral and content data to classify traffic. Each deep learning model tries to learn the data distribution in one cluster and is more effective compared to individual learning models. Qin et al. [33] have used a two-step deep learning method to detect attacks. The method can create flow rules that conform to a small number of header fields of arbitrary protocols. Dong et al. [6] propose a botnet detection model called BotDetector that can directly retrieve network stream files to extract the characteristics of botnet traffic.

Although some studies as mentioned above use raw data and automatic feature extraction, several challenges remain to be addressed. Using all the packets of a flow for intrusion detection [13, 15, 22, 45, 48], cannot provide early detection of malicious traffic. Moreover, using the payload data for traffic classification [13, 22, 45, 48, 49] can violate user privacy, while becoming impractical for encrypted traffic. Although deriving features automatically from raw data can reduce the burden of manual feature design and extraction, the detection rate of the algorithm should not be affected [15]. Due to these challenges, the current paper, we propose a deep-learning-based model for botnet detection, which uses only raw header data from initial packets of a flow, and automatically extracts features. We also use temporal information of packets and field correlations to improve classification accuracy.

3 The proposed Botnet Detection Algorithm

The goal of the proposed approach is to use the raw traffic data as input to a deep learning algorithm, to automatically learn the discriminative features for botnet classification. In this section, we first describe the high-level design of the system and then describe the details of the model.

3.1 System Model

The basic traffic units employed for botnet detection are network flows, the set of which is denoted by F . A flow contains a set of packets and each packet is combined from a set of bytes.

Definition 1 {Network Flow} A flow $f \in F$ is a packet stream $f : \{p_1^f, p_2^f, \dots\}$, distinguished by the IP addresses of the two communicating sides, the transport layer protocol, and the port numbers on each side. The flow f may belong to normal or botnet traffic. Based on the packets in the flow, a one or two-dimensional flow signature SIG_f is derived for f to be used in the botnet detection system.

Definition 2 {Network Packet} A packet $p \in f$ is a set of bytes, divided into header and payload sections. The header of each network layer consists of a set of *fields*. The capturing time of packet p is denoted as the *packet timestamp* t^p .

In this paper, we only extract bytes from the transport (TCP/UDP) and network (IP) headers. Although using the raw packet data, we only expose the header information and do not use the payload content, to increase the generality of the algorithm while preserving user privacy. The extracted bits are in the raw form and minimal preprocessing is performed for each flow. From each flow f , the first k packets are captured and transformed into the appropriate structure to form a signature for f . Relying on only the first few packets of a flow results in early detection of malicious traffic, and it can facilitate blocking of abnormal flows.

Figure 2 shows an overview of the proposed model for botnet detection. As observed, the model uses the raw traffic flows and captures the first k packets of each flow. The process involves minimal preprocessing in which some parts of the headers are discarded and two fields, namely payload size and timestamp are calculated based on the header fields and packet capturing metadata, respectively. In the preprocessing step, a bit-stream is calculated for each packet, represented as a vector of zero/one values and named here as *packet vector*. After that, a *correlation vector* is computed for a flow based on the packet vectors in that flow. Next, we generate a *flow signature* for each flow. We propose four types of flow signature in this paper and evaluate the performance of our approach based on these signatures. More details about constructing the flow signatures are presented in Sect. 3.3. In the next step, the flow signatures are given to the LSTM network. After training the LSTM network, the feature vector of each flow is extracted from the hidden layer of LSTM, and given as input to a classifier. The model steps are described in detail in the next subsections. The notations used in this paper along with their descriptions are summarized in Table 1.

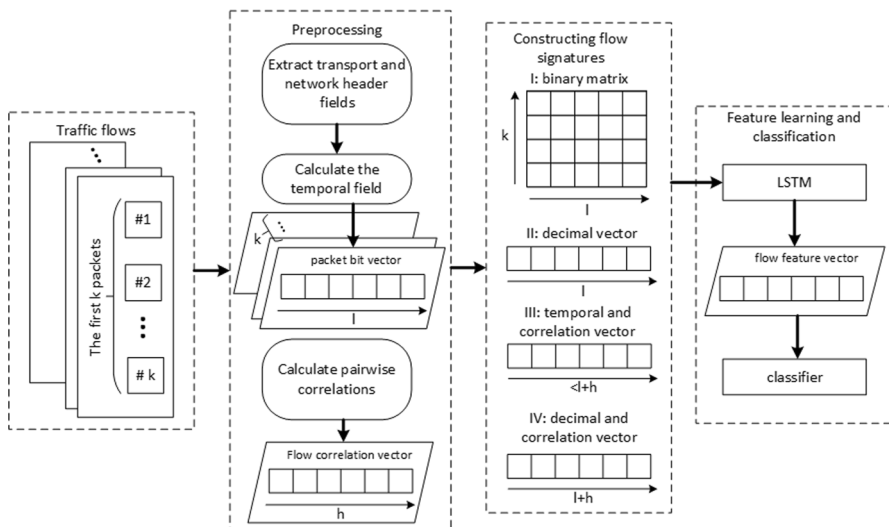


Fig. 2 The process of the proposed botnet detection approach

Table 1 Notations used in this paper

Parameter	Description
F, f	A batch of received flows, one typical flow in a batch
hf_i^p, m	The i th header field of packet p , the total number of header fields
v^p, l	The binary vector of packet p , its length in bits
t^p	Capturing time of packet p
cv^f, h	The correlation vector of flow f , its length
sig_f^*	Signature of a flow f

3.2 Preprocessing

In what follows, we first describe the raw data extracted from each packet and then explain the derivative information which may be employed in constructing the flow signature.

3.2.1 Header Fields

From the captured packets, multiple fields of the IP and TCP/UDP headers are extracted as described in Table 2. All the fields have their original raw values from the packet headers, except the last two fields. The payload length is computed for both the TCP and UDP packets from the appropriate fields of transport and network headers. The timestamp t^p of packet p is the capturing time of this packet and is acquired

Table 2 The fields extracted from packet headers

Protocol	Field	Source	Size (bits)
TCP	Sequence Number	TCP header	32
	Acknowledgment Number		32
	Data Offset		6
	Flags		8
	Window size		16
	Urgent pointer		16
TCP/UDP	Source port	TCP/UDP header	16
	Destination port		16
	Checksum		16
	Version	IP header	4
	Type of service		8
	Time to live		8
	Protocol		8
	Checksum	Calculated	16
	PayLoadLength		Max required
	TempDiff		Max required (64)

from the packet analyzer. To obtain relevant information about the temporal aspects of a flow, we consider the differences of the packet timestamps instead of the actual timestamp values. These intervals provide more insight into the timing of packets in a flow and are more informative than the raw timestamps.

Definition 3 *{Temporal Difference}* Let p and p' denote two consecutive packets in a flow. The temporal difference field for p is computed by Eq. (1), and for the k^{th} packet of the flow, $tempDiff$ is set to zero.

$$tempDiff_p = t^p - t^{p'} \quad (1)$$

The total number of fields extracted or derived for a packet is denoted by m . Note that m is different based on the transport protocol (16 for TCP and 10 for UDP). The header fields mentioned in Table 2 for a packet p are denoted as $\{hf_i^p\}_{1 \leq i \leq m}$ regardless of the network layer and whether the field is raw or derived. The length of each field is also mentioned in Table 2. The two last fields are stored in the maximum required bits according to the largest values among all packets. Zero padding is used to make the field lengths equal among all packets.

Definition 4 *{Packet Vector}* For a packet p a corresponding one-dimensional binary packet vector v^p of length l bits is constructed by concatenating the bit values from all header and derivative fields, $v^p : \langle hf_1^p, \dots, hf_m^p \rangle$. Note that l is different for the TCP and UDP packets.

3.2.2 Correlation Vector

The correlation between pairs of packet fields in a flow may expose additional dependency information to improve the performance of the deep network. The correlation is computed between fields of the first k packets in a flow f . To this end, we first compute m decimal attributes $\{a_j^f\}_{1 \leq j \leq m}$, for f . To do so, we consider the separate header fields of each packet, as binary numbers. In Eq. (2), for each such field, the corresponding values in the first k packets of f are added together, and the result is converted to a decimal number.

$$a_j^f = \left(\sum_{i=1}^k \left(hf_j^{p_i^f} \right)_b \right)_d \quad (2)$$

The subscripts b and d in Eq. (2) emphasize whether the values are binary or decimal, respectively. The decimal attributes are normalized using the standard score shown in Eq. (3):

$$\tilde{a}_j^f = \frac{a_j^f - \bar{a}_j}{\sigma_{a_j}} \quad (3)$$

where $\bar{a}_j$ and σ_{a_j} denote the mean and standard deviation of the j^{th} attribute among all flows, respectively. After normalization, the pairwise correlation value is computed for each flow independently. This pairwise correlation can provide a better understanding of the dependency among different fields in a flow. With a more accurate insight into the flow, the learning algorithm provides a higher performance in both learning the data and deciding on unseen samples. We adapt the correlation calculation from [3] to compute the pairwise correlations in a flow f using Eq. (4).

$$corr^f(i, j) = \frac{\tilde{a}_i^f \times \tilde{a}_j^f}{2} \quad (4)$$

Definition 5 {Flow Correlation Vector} For each flow f , a number of $\binom{m}{2}$ pairwise correlation values are generated to form a one-dimensional decimal correlation vector $\mathbf{cv}^f : < corr^f(i, j) >, 1 < i, j < m$ of length h .

3.3 Flow Signatures

The basic units analyzed for botnet detection are network flows. We produce a signature for each flow and provide the signatures as inputs to the deep learning network. The signatures are constructed based on the packet and flow vectors described in the previous section. We propose four different one or two-dimensional signatures for each flow, which enable analyzing the effect of the various configurations of features. However, it is worth noting that a flow signature is produced based on the raw data of that flow, with minimal preprocessing to provide more insight for the deep network or reduce the signature size. The main source of feature extraction is the packet vector itself, which is given in its raw format to the deep network. In addition, to improve the computational overhead by providing one-dimensional inputs to the network, we propose minimal preprocessing using operators such as count and addition. The raw header bytes however, do not provide information on timing of the packets. Therefore, the temporal information is included in some constructed signatures. Another knowledge shown to be beneficial in classification is the correlation of input features. As we do not extract features manually, correlation is computed for the actual header fields. The two latter parts of information are shown to be effective for improving the botnet detection performance in the experiments.

Signature I includes the complete raw bytes of a flow (with the addition of temporal difference and size fields) and does not contain the correlation information. It can evaluate the power of deep learning in extracting features when no additional information is provided.

Definition 6 {Signature I ($\mathbf{sig}_I^f$)} For each flow f , a $k \times l$ binary matrix is created in which the i^{th} row of the matrix is the packet vector of the i^{th} packet in the flow.

Therefore, we have $\mathbf{sig}_I^f \equiv \begin{bmatrix} -\mathbf{p}_1^f - \\ \dots \\ -\mathbf{p}_k^f - \end{bmatrix}$.

In contrast to signature I, which was two-dimensional, the signatures II-IV are one-dimensional vectors. The second signature is again based on complete raw data as in signature I, but reduces the computational overhead for training the LSTM network by converting the matrix into a one-dimensional vector.

Definition 7 {Signature II (sig_{II}^f)}. For each flow f , a column-wise counting of ones is applied to sig_I^f to convert the $k \times l$ matrix into a decimal vector of length l with each entry ranging from zero to k . The second signature is constructed based on this vector, $\text{sig}_{II}^f \equiv \langle e_1, \dots, e_l \rangle$, where $0 \leq e_i \leq k$ is the number ones in the i th column of the sig_I^f matrix.

The third signature shows if botnet detection is feasible based on only temporal information and field dependencies of a flow, omitting the actual raw values of packet headers. To construct the third signature, it is required to split the vector of sig_{II}^f based on the underlying header fields and extract the part related to the temporal difference field, which is the last field in the packet vector. Therefore, the vector of the second signature is restarted based on the underlying fields, as follows where the values related to each field are specified:

$$\text{sig}_{II}^f \equiv \left\langle \underbrace{e_1, \dots, e_i}_{hf_1}, \underbrace{e_{i+1}, \dots, e_{i'}}_{hf_2}, \dots, \underbrace{e_{i''}, \dots, e_l}_{hf_m \equiv \text{tempdiff}_d^f} \right\rangle.$$

The values declared as tempdiff_d^f are related to the temporal difference field in sig_{II}^f , and the subscript d emphasizes the values are decimal.

Definition 8 {Signature III (sig_{III}^f)}. For each flow f , the entries related to the temporal difference field in sig_{II}^f are extracted as tempDiff_d^f , and concatenated to the correlation vector of flow f to form the third signature, $\text{sig}_{III}^f \equiv \langle \text{tempDiff}_d^f, \text{cv}^f \rangle$.

The final and fourth signature has raw data of the flow along with field correlations and is the most complete signature.

Definition 9 {Signature IV (sig_{IV}^f)}. For each flow f , sig_{II}^f is concatenated to the correlation vector cv^f , to construct the fourth signature, i.e. $\text{sig}_{IV}^f \equiv \langle \text{sig}_{II}^f, \text{cv}^f \rangle$.

3.4 Feature Learning and Classification

The RNN [14] is a deep structure developed from the feedforward neural network and has an internal state enabling it to learn from variable length input sequences. It can detect patterns in streaming data. The design of internal connections in an RNN allows considering temporal relations, such that the output of each time step is dependent on the outputs of the previous states. Accordingly, what the network has learned in the past, has an impact on the future outcome. The LSTM [14] is the main variant of RNNs which is commonly applied in different domains. In RNN, the input and the hidden states are passed through a single $\tanh$ layer. LSTM introduces additional gates and a cell state as follows: The forget gate (ϕ) controls persistence

of a value in the cell, the input gate (i) restrains the entrance of a new value into the cell, and the output gate (o) determines how much effect the cell value has on the cell output.

The hidden layer of LSTM consists of a set of recurrently connected units. The input vector x^t is the deep network input at time t . Elements of each block in a layer of memory cells are defined by the set of Eqs. (5) [14]. g is the block input at time t , and along with the input gate, they decide on the new information that should be stored in the cell state s . h_{l-1}^t shows the previous layer (or network input) at the same step t , and h_l^{t-1} is the same layer at the previous step $t-1$. h_l^t is the block output at time t . In addition, w is the weight parameter, and b is the bias. The σ and $\tanh$ are point-wise sigmoid (logistic) and hyperbolic tangent activations, respectively. The $\odot$ operator symbolizes point-wise multiplication

$$\begin{aligned}\varphi_l^t &= \sigma(w_l^{fx} h_{l-1}^t + w_l^{fh} h_l^{t-1} + b_l^f) \\ i_l^t &= \sigma(w_l^{ix} h_{l-1}^t + w_l^{ih} h_l^{t-1} + b_l^i) \\ o_l^t &= \sigma(w_l^{ox} h_{l-1}^t + w_l^{oh} h_l^{t-1} + b_l^o) \\ g_l^t &= \tanh(w_l^{gx} h_{l-1}^t + w_l^{gh} h_l^{t-1} + b_l^g) \\ s_l^t &= g_l^t \odot i_l^t + s_l^{t-1} \odot \varphi_l^t \\ h_l^t &= \tanh(s_l^t) \odot o_l^t\end{aligned}\quad (5)$$

An LSTM network is used to extract features from the flow signatures. The signatures, as described in the previous section, contain mostly raw data bytes or are constructed from raw data with minimal preprocessing. Therefore, the proposed model can classify flows with less computational overhead. When adopting the first signature, the matrix is inserted into the LSTM network in k consecutive steps for each input flow. The next three signatures which are vectors are given as a multidimensional record in one step to the network. After training the LSTM network, the extracted features for the flows are used to train a classifier for classifying the abnormal traffic flows. Different classifiers may be used in the last step to detect botnets. Specifically, the LSTM network itself may act as the detection engine with the last sigmoid layer performing the classification. Alternatively, the features of the flows may be extracted from the LSTM layer preceding the sigmoid layer and applied to a standalone classifier. This classifier should be first trained with the known normal and botnet flows to be used as the detector of unknown flows. In this paper, we adopt both approaches of classification and compare their performance.

4 Experiments

In this section, we present the experimental results of the proposed approach and also compare the performance with several recent studies [2, 4, 15, 16, 47]. We also show that botnet detection based on raw flow data can be more effective

compared to feature engineering. In what follows, we describe the dataset the evaluation model and then move on to presenting the actual results.

4.1 Dataset

A reliable assessment of botnet detection systems requires a comprehensive data set that includes a variety of botnets with different protocols. On the other hand, the design of a botnet detection system requires the presence of unseen samples in the test set, so that the efficiency of the system can be measured for detecting different types of new botnets.

This paper uses the ISCX¹ [37] data set, which has so far been the most comprehensive dataset among those related to botnets, and has a high level of diversity from different types of botnets. The ISCX dataset is developed at CIC Laboratory at UNB University. Because this data set provides heterogeneous traffic flows, it can demonstrate the real-time simulation of network traffic and enable a more reliable assessment of the proposed system. The dataset includes two subsets of training and test data. The training dataset includes seven different types of botnets with a volume of 5.3 GB, of which 92.43% are malicious. The test dataset has 16 botnet types with a volume of 8.5 GB, 44.97% of which are malicious flows. As the dataset contains unseen botnet types in the test set, it can evaluate the performance of the model in detecting previously unseen botnet patterns.

The raw data from the ISCX data set were extracted by the CICFlowMeter [26] tool. For the experiments, only the first 10 packets of each flow were extracted ($k = 10$). This allows early detection of security attacks, compared to when all packets in the flow should be examined, and decreases the damage caused by the attacks. If the number of packets is less than 10 for a flow, zero-bit packets are inserted accordingly.

4.2 Model Configuration

The implementation of the LSTM model was performed using the Keras [12] library with the TensorFlow [1] backend. We used the Adam optimizer [25], with a learning rate of 0.01. The binary cross-entropy is used as the error function. The network specifications are tuned experimentally. For the TCP flows, a three-layer LSTM is used with 400, 300, and 55 units in the consecutive layers respectively. For the UDP flows, numbers of units are 300, 200, and 65 respectively. The last dense layer has one sigmoid unit. The activation of the first layer in both architectures is ReLU, and the next layers have sigmoid activation. The dropout rates for the second and third layers are 0.5 and 0.2, respectively.

For the final classifier, we either use the LSTM network itself, or a Random Forest (RF) which is a widely used learning approach that has demonstrated good accuracy in botnet detection studies [26, 36].

¹ <http://www.unb.ca/cic/datasets/botnet.html>

4.3 Evaluation Metrics

The performance of the proposed method is evaluated by four criteria as described in this section. The accuracy metric [18] shows the portion of flows that are classified correctly. Precision [18] and recall [18] are used simultaneously to measure the classification performance. Precision indicates the capability of the classifier to predict actual attacks with concerning to all the predicted attack flows, while recall refers to the percentage of predicted attack flows relative to all attack flows. Higher values of these two measures (in the ideal case equal to one) indicate a better classifier performance. Therefore, if precision and recall are high for a class, it means that the classifier performs well in identifying that category. These criteria are described in Eqs. (6), (7) and (8).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (6)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (7)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (8)$$

In a two-class scenario (positive and negative), the number of positive/negative predictions that are correctly classified is called True Positive (TP)/True Negative (TN). In contrast, False Positive (FP)/False Negative (FN) indicates the number of actual negative/positive samples classified as positive/negative.

Another popular performance measurement is F1-Score [18]. The F1-Score value, as shown in Eq. (9), is between zero and one and performs a better evaluation by combining precision and recall. A high F1-Score indicates the simultaneous efficiency of both Precision and Recall. If the precision and recall values are exactly one, the F1-Score value also becomes one.

$$\text{F1 - Score} = \frac{2 \times \text{precision} \times \text{recall}}{\text{Precision} + \text{recall}} \quad (9)$$

Another criterion for evaluating performance is the AUC-ROC curve [18]. This curve shows the model's capability in discriminating the classes. To plot the ROC curve, the discrimination threshold of the classifier is varied. The TPR (True Positive Rate or Sensitivity), which is similar to recall, is plotted against FPR (False Positive Rate or 1-Specificity). Sensitivity [18] and specificity [18], which are inversely proportional, are two criteria used together to measure the predictive performance of a classification model. These criteria are described in Eqs. (10), (11) and, (12).

$$\text{TPR} = \frac{TP}{TP + FN} \quad (10)$$

$$\text{Specificity} = \frac{TN}{TN + FP} \quad (11)$$

$$\text{FPR} = 1 - \text{Specificity} = \frac{FP}{FP + TN} \quad (12)$$

4.4 Results

4.4.1 Performance Evaluation

The first set of experiments reveal the model performance in detecting botnet and benign flows. The ISCX training set is used for model construction and the evaluation metrics are reported on the test set, averaged across TCP and UDP flows. As described previously, four different signatures may be extracted for each flow. We compare the detection rate based on these signatures in Fig. 3. As described before, the final classifier is either the LSTM or RF. According to the results, the second signature yields the least metric results. This shows that summarizing the raw flow data to decrease the classification complexity, has adversely affected the model accuracy. The third signature offers better results, but including only temporal and correlation information, it cannot benefit from header contents. The first signature, which has complete raw data without summarization has the second-best results. This shows the raw data can be used to automatically extract discriminative features and manual feature extraction can be eliminated. The fourth experiment offers the best classification results as it has extra information on field correlations in each flow.

Table 3 shows the results of the same experiment, based on precision, recall, and F-measure metrics where similar trend is observed for different signatures. Overall, it is observed that the deep learning algorithm has been able to derive the essential features from raw flow data. In addition, the RF classifier improves the classification results compared to the sole LSTM. Similar experiments were performed without the temporal difference field, and the accuracies for all signatures dropped by almost

Fig. 3 The proposed model accuracy compared for the four signatures

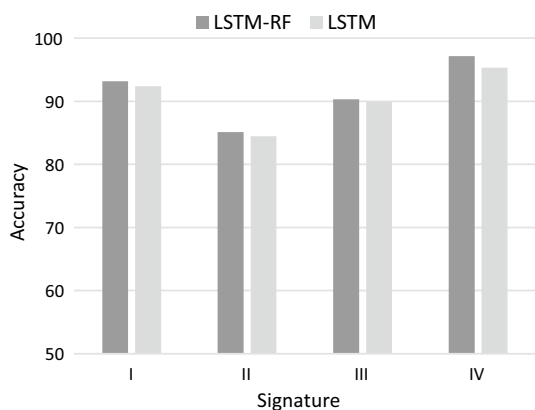


Table 3 Evaluation of the proposed model on the ISCX test dataset

Signature	Precision (%)		Recall (%)		F-Score (%)	
	LSTM-RF	LSTM	LSTM-RF	LSTM	LSTM-RF	LSTM
I	92	92	94	92	92.98	92
II	86	85	85	84	85.49	84.5
III	90	90	89	90	89.5	90
IV	98	95	98	94	98	94.5

Best values are indicated in bold

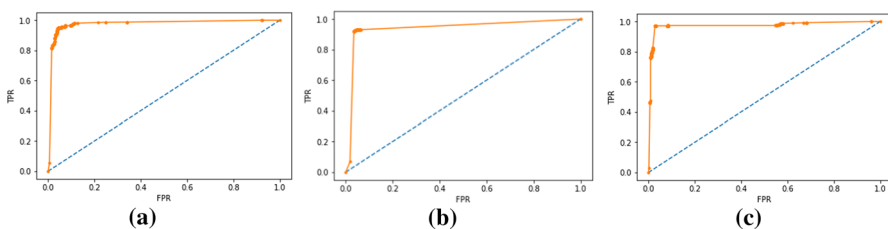
40%. This observation stresses the importance of temporal information in detecting abnormal behavior of network traffic. A considerable portion of the test dataset contains botnet samples that are not observed in the training dataset. This shows that the model can detect unseen botnet types.

Figures 4 involve the ROC curves for TCP, UDP, and all flows in the test data. It can be seen that the algorithm performance is slightly higher for TCP flows compared to the UDP flows. The ROC curve for the integration of the TCP and UDP flows shows that the algorithm is successful in classifying normal traffic from botnet flows. As observed, the TPR value is close to one where the FPR value is very low and less than 0.1. This shows that the algorithm can correctly detect botnet traffic, while keeping the false alarm rates low at the same time.

4.4.2 Evaluating the Use of Raw Data

To show that using raw flow data for botnet detection is more effective than manual feature extraction, we conduct a study in which several feature sets are extracted from the ISCX dataset and used for classification. We adopt the feature sets shown to be effective in botnet detection from previous studies. It is important to note that the mentioned studies do not use the ISCX dataset in their original evaluations, and thus may report different results compared to the values presented here. This variation is previously observed in the literature [2] where evaluations on datasets with different diversity of botnets have largely affected the algorithm outcome.

The first feature set consists of four features introduced by Lashkari et al. [26] which are selected based on the C4.5 classifier [34]. They have originally used a Tor

**Fig. 4** ROC curve for **a** TCP, **b** UDP, and **c** all flow in test data

traffic dataset for evaluation. The second feature set consists of 10 features, which are selected as the most discriminative ones among 248 features by Celik et al. [5]. They have performed simulations on three datasets not including ISCX. The third feature set consists of 11 discriminative features selected using the C4.5 and Ada-boost algorithms from 248 features by Li et al. [27]. They examine network data of two different sites. Finally, the fourth feature set consists of four features chosen by the RF algorithm by Sharafaldin et al. [36]. They originally used the CIC2018 dataset. The description of the features is available in Table 4. The LSTM architecture used for botnet detection based on features is chosen experimentally and consists of three hidden layers and one dense layer, having 120, 64, 32, and one unit respectively. The first layer has ReLU activation and the other activations are sigmoid.

Table 5 shows the accuracy metric for different feature sets using the ISCX dataset. As observed, the third and fourth feature sets yield the best performance among the four. To further extend and improve the feature-based botnet detection, we combine the two latter feature sets into 16 features and observe that it can produce about 9% higher detection accuracy. Still trying to improve the results, we refer to the previous section in which we showed that the correlation information can improve the performance of botnet detection by providing more insight into feature dependencies. Therefore, another feature set was constructed in which we added the mutual correlations of the 16 features to the dataset. This last feature set produced the highest detection accuracy when using feature engineering.

The previous results were based on the LSTM classifier. The experiment is further expanded by adding two other popular machine-learning algorithms, the C4.5 decision tree and, the Naïve Bayes classifier. The two classifiers are used for botnet detection using the last feature set (feature sets 3, 4, and correlation). We compare the performance of these methods (LSTM, C4.5, Naïve Bayes based on the last feature set) with our proposed approach of using raw flow data in Fig. 5. As observed, none of the schemes can surpass the accuracy of the proposed approach, which offers about a 7.5% higher detection rate. This experiment exposed the efficiency of using raw network flow data compared to feature engineering for botnet detection.

4.4.3 Comparison

To show the superiority of the proposed approach compared to other studies using the ISCX data set, a comparison is performed in Table 6. In the proposed algorithm, the fourth signature is used for the network input as it offered the best accuracy values in previous experiments. The RF classifier is adopted to detect botnet flows. All the included studies have used the ISCX training dataset for modeling the traffic flows, and the reported detection rates are based on the test dataset.

Among the compared approaches, Yin et al. [47] use a generative adversarial network. Beigi et al. [2] use a set of manually designed features for botnet detection with the C4.5 decision tree. Homayoun et al. [15] use a deep structure composed of an autoencoder and CNN. Qin et al. [33] use a two-step deep learning method to detect attacks. It can create flow rules that conform to a small number of header fields of arbitrary protocols. Bijalwan et al. [4] use an ensemble of classifiers for botnet detection. Hossain et al. [16] first reduce the manually selected features and

Table 4 Description of manually extracted features used for comparison

Feature	Description	Feature set 1 [26]	Feature set 2 [5]	Feature set 3 [27]	Feature set 4 [36]
Flow duration	Difference between last packet time and first packet time (μ_s)	✓	✓		✓
IOPR	The ratio between the number of packets received and the number of packets sent	✓			
APL	Average length of packets	✓			
BS	Average bits per second (packet speed b/s)	✓			
Cnt_data_pkt	Count of all the packets with at least one byte of payload (forwarding)		✓		
Min_data_size	Minimum payload size observed (forwarding)		✓		
Mean_Bytes	Data bytes divided by the total number of packets (backwarding)		✓		
Int_data_Len	The total number of bytes sent before the first ACK packet was observed (bidirectional)		✓		
RTT_samples	Total number of RTT samples found in total packets (forwarding)		✓	✓	
Med_Bytes	The median byte length of packets (forwarding)		✓		
Var_Bytes	The variance of byte length of packets (backwarding)		✓	✓	
IP_ratio	The ratio between the maximum packet size and minimum packet size (bidirectional)		✓		
Goodput	Total number of frame bytes divided by flow duration (bidirectional)		✓		
Push_pkt_serv	Count of all packets with push bit set in TCP header (backwarding)			✓	
Int_win_bytes	The total number of bytes sent in the initial window (bidirectional)			✓	
Avg_seg_size	Average segment size: data bytes divided by #packets (backwarding)			✓	
IP bytes med clnt	Median of total bytes in IP packet (forwarding)			✓	
Act_data_pkt	Count of packets with at least 1 byte of TCP data payload (forwarding)			✓	
Min_seg_size	Minimum segment size observed (forwarding)			✓	
Push_pkt_clnt	Count of all packets with push bit set in TCP header (forwarding)			✓	
Serv_port	Server port			✓	
Clnt_port	Client port			✓	
Total Len F.Packets	The entire length of the packets (forwarding)				✓

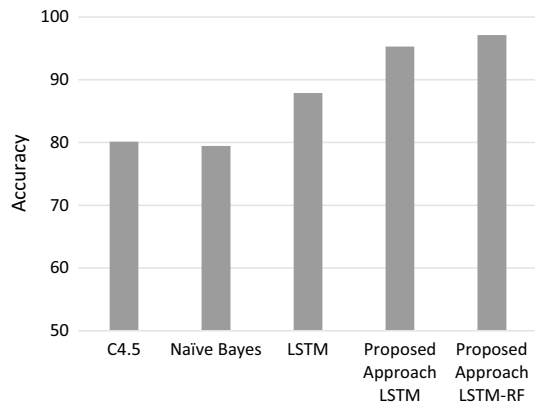
Table 4 (continued)

Feature	Description	Feature set 1 [26]	Feature set 2 [5]	Feature set 3 [27]	Feature set 4 [36]
SubFlow F.Bytes	Number of bytes in the sub-flow (forwarding)				✓
B.Packers/s	Number of backward packets per second (p/s) (backwarding)				✓

Table 5 The accuracy of LSTM network on different feature sets using the ISCX test set

Feature set	Accuracy
Feature set 1 [26] (LSTM)	67.12
Feature set 2 [5] (LSTM)	53.20
Feature set 3 [27] (LSTM)	70.01
Feature set 4 [36] (LSTM)	72.65
Feature set 3, 4 (LSTM)	81.90
Feature set 3, 4, correlation (LSTM)	87.88

Best value is indicated in bold

Fig. 5 Comparing detection accuracy of feature-based classification with the proposed approach**Table 6** Comparison with other studies on ISCX 2014 dataset

Method	Accuracy	FPR	TPR	F-Score
Yin et al. [47]	71.17	15.59	–	–
Beigi et al. [2]	75	2.3	–	–
Homayoun et al. [15]	–	15	92	–
Hossain et al. [16]	86.41	–	–	–
Qin et al. [33]	91.1	–	–	86.1
Bijalwan et al. [4]	96.41	–	–	–
Proposed method (LSTM-RF, Sig IV)	97.13	14	98.18	98

then use deep learning for botnet detection. All these methods use predefined features from all packets in a flow as inputs to the learning algorithm. To the best of our knowledge, no previous work on the ISCX dataset uses raw flow data as the model input. As it was previously shown in the experiments, using raw flow data along with temporal and correlation information can largely increase the algorithm's accuracy in detecting botnets. The performance improvement is notable compared to some studies using manual feature engineering or traditional classifiers in Table 6. Manual features may limit the knowledge required for accurate performance of the classifiers. In addition, some recent proposals have omitted the important FPR measure in

their reports. The proposed approach has the lowest FPR rate after the work of Beigi et al. [2]. Their approach has been able to provide the lowest false alarms, but it cannot produce a simultaneous high detection rate and misses many of the botnet flows.

In another study not using the ISCX dataset for evaluation, Dong et al. [6] proposed the BotDetector that can directly retrieve network stream files to extract the characteristics of botnet traffic. However, in contrast to our study, they used both the header and payload parts of the packets which may violate data privacy. Some other studies use manual feature selection from network flows. Garre et al. [48] and Jagadeesan et al. [20] used different machine learning methods for detecting botnets. Hosseini et al. [17] used deep learning methods to select the appropriate features of network flows. Various types of deep learning methods have been used to classify and extract features in other studies [11, 21, 29]. In the mentioned methods, after manual feature extraction, the learning algorithm is trained for classification. Manual feature selection requires trial and error experiments, and these approaches may not be generalized for different botnet types and traffic flows. Overall, it is observed that the proposed method in this study can be efficiently used for representation learning in botnet detection, extracting features from the initial packets of a flow while preserving the privacy of packet payload, omitting the feature engineering step, and detect botnet flows with high accuracy.

5 Conclusion

In this paper, we proposed a deep-learning approach for botnet detection, in which the deep neural network automatically learns the characteristics of raw network data. The proposed approach extracts the features only from the headers of the network packets. The timestamp of the packets is also used as very important data in the feature learning phase. To detect an attack in the nearest moments of starting the attack process, we only use the first few packets in each network flow for testing the system. The proposed preprocessing techniques, such as calculating the correlation relationship between the header fields, made our approach more accurate compared to the state-of-the-art techniques. We provided a comparison with several feature-engineering-based botnet detection algorithms to show the effectiveness of using raw traffic data. The FPR of a botnet detection system is an important feature, which affects the unnecessary burden of false security alarms. Further research is required to reach a smaller FPR. Also, based on the recent advances in applying deep learning time series classification, further research can be performed for increasing the system performance.

References

1. Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M.: Tensorflow: A system for large-scale machine learning. In: 12th {USENIX} symposium on operating systems design and implementation ({OSDI} 16). Pp. 265–283 (2016)

2. Beigi, E.B., Jazi, H.H., Stakhanova, N., Ghorbani, A.: A Towards effective feature selection in machine learning-based botnet detection approaches. In: 2014 IEEE Conference on Communications and Network Security. IEEE, pp. 247–255 (2014)
3. Bengio, Y., Courville, A., Vincent, P.: Representation learning: A review and new perspectives. *IEEE Trans. Pattern Anal. Mach. Intell.* **35**(8), 1798–1828 (2013)
4. Bijalwan, A., Chand, N., Pilli, E.S., Krishna, C.R.: Botnet analysis using ensemble classifier. *Perspect. Sci.* **8**, 502–504 (2016)
5. Celik, Z.B., Walls, R.J., McDaniel, P., Swami, A. Malware traffic detection using tamper-resistant features. In: MILCOM 2015–2015 IEEE Military Communications Conference, pp. 330–335 (2015)
6. Dong, X., Dong, C., Chen, Z., Cheng, Y., Chen, B.: BotDetector: An extreme learning machine-based Internet of Things botnet detection model. *Trans. Emerg. Telecommun. Technol.* **32**(5), e3999 (2021)
7. Eesa, A.S., Orman, Z., Brifcani, A.M.A.: A novel feature-selection approach based on the cuttlefish optimization algorithm for intrusion detection systems. *Expert Syst. Appl.* **42**(5), 2670–2679 (2015)
8. Felix, J., Joseph, C., Ghorbani, A.: A Group behavior metrics for P2P Botnet detection. In: International Conference on Information and Communications Security, Springer, 93–104 (2012)
9. Garre, J.T.M., Pérez, M.G., Ruiz-Martínez, A.: A novel Machine Learning-based approach for the detection of SSH botnet infection. *Future Gener. Comput. Syst.* **115**, 387–396 (2021)
10. Goldberg, Y.: A primer on neural network models for natural language processing. *J. Artif. Intell. Res.* **57**, 345–420 (2016)
11. Goodfellow, I., Bengio, Y., Courville, A., Bengio, Y.: *Deep Learning*, vol. 1. MIT Press, Cambridge (2016)
12. Gulli, A., Pal, S.: *Deep Learning with Keras*. Packt Publishing Ltd, Birmingham (2017)
13. Hamed, T., Dara, R., Kremer, S.C.: Network intrusion detection system based on recursive feature addition and bigram technique. *Comput. Secur.* **73**, 137–155 (2018)
14. Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Comput.* **9**(8), 1735–1780 (1997)
15. Homayoun, S., Ahmadzadeh, M., Hashemi, S., Dehghantanha, A., Khayami, R.: BoTShark: A deep learning approach for botnet traffic detection. In: *Cyber Threat Intelligence*, pp. 137–153. Springer, New York (2018)
16. Hossain, M. I., Eshrak, S., Auvik, M. J., Nasim, S. F., Rab, R., Rahman, A.: Efficient Feature Selection for Detecting Botnets based on Network Traffic and Behavior Analysis .In 7th International Conference on Networking, Systems and Security, pp. 56–62 (2020).
17. Hosseini, S., Nezhad, A.E., Seilani, H.: Botnet detection using negative selection algorithm, convolution neural network and classification methods. *Evol. Syst.* **13**, 1–15 (2021)
18. Hossin, M., Sulaiman, M.N.: A review on evaluation metrics for data classification evaluations. *Int. J Data Mining Knowl. Manag. Process* **5**(2), 1 (2015)
19. Hubballi, N., Suryanarayanan, V.: False alarm minimization techniques in signature-based intrusion detection systems: A survey. *Comput. Commun.* **49**, 1–17 (2014)
20. Jagadeesan, S., Amutha, B.: An efficient botnet detection with the enhanced support vector neural network. *Measurement* **176**, 109140 (2021)
21. Javid, A., Niyaz, Q., Sun, W., Alam, M.: A deep learning approach for network intrusion detection system. In: *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies (formerly BIONETICS)*, 21–26 (2016)
22. Khan, M.A., Karim, M., Kim, Y.: A scalable and hybrid intrusion detection system based on the convolutional-LSTM network. *Symmetry* **11**(4), 583 (2019)
23. Kheir, N., Wolley, C.: Botsuer: Suing stealthy p2p bots in network traffic through netflow analysis. In: *International Conference on Cryptology and Network Security*, Springer, 162–178 (2013)
24. Kim, K.: Aminanto ME Deep learning in intrusion detection perspective: Overview and further challenges. In: *2017 International Workshop on Big Data and Information Security (IWBIS)*. IEEE, 5–10 (2017)
25. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. <http://arxiv.org/abs/1412.6980> (2014)
26. Lashkari, A.H., Draper-Gil, G., Mamun, M.S.I., Ghorbani, A.: A Characterization of tor traffic using time based features. In: *ICISSp*, 253–262 (2017)
27. Li, W., Canini, M., Moore, A.W., Bolla, R.: Efficient application identification and the temporal and spatial stability of classification schema. *Comput. Netw.* **53**(6), 790–809 (2009)
28. Liao, W.-H., Chang, C.-C.: Peer to peer botnet detection using data mining scheme. In: *2010 International Conference on Internet Technology and Applications*, IEEE, 1–4 (2010)

29. Ma, T., Wang, F., Cheng, J., Yu, Y., Chen, X.: A hybrid spectral clustering and deep neural network ensemble algorithm for intrusion detection in sensor networks. *Sensors* **16**(10), 1701 (2016)
30. Meghdouri, F., Vázquez, F. I., & Zseby, T. (2020). Cross-Layer Profiling of Encrypted Network Data for Anomaly Detection. In *2020 IEEE 7th International Conference on Data Science and Advanced Analytics (DSAA)* (469–478). IEEE.
31. Miller, S., Busby-Earle, C.: The impact of different botnet flow feature subsets on prediction accuracy using supervised and unsupervised learning methods. *Int. J. Internet Technol. Secur. Trans.* **5**(2), 474–485 (2016)
32. Milosevic, N., Dehghantanha, A., Choo, K.-K.R.: Machine learning aided Android malware classification. *Comput. Electr. Eng.* **61**, 266–274 (2017)
33. Qin, Q., Poularakis, K., & Tassioulas, L.: A Learning Approach with Programmable Data Plane towards IoT Security. In *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*, 410–420 (2020)
34. Quinlan, J.: *C4.5: Programs for Machine Learning*. Morgan Kaufmann Publishers, Burlington (2014)
35. Saad S, Traore I, Ghorbani A, Sayed B, Zhao D, Lu W, Felix J, Hakimian P (2011) Detecting P2P botnets through network behavior analysis and machine learning. In: 2011 Ninth annual international conference on privacy, security and trust, IEEE, 174–180
36. Sharafaldin I, Lashkari AH, Ghorbani A (2018) A Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: *ICISSP*, 108–116
37. Shiravi A, Shiravi H, Tavallaee M, Ghorbani AA (2012) Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *computers & security* **31** (3):357–374
38. Silva, S.S., Silva, R.M., Pinto, R.C., Salles, R.M.: Botnets: A survey. *Comput. Netw.* **57**(2), 378–403 (2013)
39. Singla, A., Bertino, E.: How deep learning is making information security more intelligent. *IEEE Secur. Priv.* **17**(3), 56–65 (2019)
40. Tan, Z., Jamdagni, A., He, X., Nanda, P., Liu, R.P.: A system for denial-of-service attack detection based on multivariate correlation analysis. *IEEE Trans. Parallel Distrib. Syst.* **25**(2), 447–456 (2013)
41. Tegeler F, Fu X, Vigna G, Kruegel C (2012) Botfinder: Finding bots in network traffic without deep packet inspection. In: *Proceedings of the 8th international conference on Emerging networking experiments and technologies*, 349–360
42. Van NT, Thinh TN (2017) An anomaly-based network intrusion detection system using deep learning. In: *2017 International Conference on System Science and Engineering (ICSSE)*, IEEE, 210–214
43. Vormayr, G., Zseby, T., Fabini, J.: Botnet communication patterns. *IEEE Commun. Surv. Tutor.* **19**(4), 2768–2796 (2017)
44. Wang W, Fang B, Zhang Z, Li C (2009) A novel approach to detect IRC-based botnets. In: *2009 International Conference on Networks Security, Wireless Communications and Trusted Computing*, IEEE, 408–411
45. Wang, W., Sheng, Y., Wang, J., Zeng, X., Ye, X., Huang, Y., Zhu, M.: HAST-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection. *IEEE Access* **6**, 1792–1806 (2017)
46. Yahyazadeh, M., Abadi, M.: BotGrab: A negative reputation system for botnet detection. *Comput. Electr. Eng.* **41**, 68–85 (2015)
47. Yin C, Zhu Y, Liu S, Fei J, Zhang H (2018) An enhancing framework for botnet detection using generative adversarial networks. In: *2018 International Conference on Artificial Intelligence and Big Data (ICAIBD)*, IEEE, 228–234
48. Yu Y, Long J, Cai Z (2017) Session-based network intrusion detection using a deep learning architecture. In: *International Conference on Modeling Decisions for Artificial Intelligence*, Springer, 144–155
49. Zhong, W., Yu, N., Ai, C.: Applying big data based deep learning system to intrusion detection. *Big Data Mining Anal.* **3**(3), 181–195 (2020)